

Relatório de Desenvolvimento

Sumário

1. Introdução.....	2
2. Contextualização do Problema	2
2.1 Redundancia de Nomenclaturas	2
Solução.....	3
2.2 Mais de 1 exame na mesma linha.....	3
Solução	3
2.3 Grande quantidade de alergenoss.....	3
Solução.....	4
3. Fluxo	4
4. Extração e Tratamento	6
3.1 Extração Saúde	6
3.2 Normalização desse DataFrame:	7
3.3 Extração Clínica.....	8
5. Verificação paciente faturado x solicitações	9
6. Comparação dos exames.....	10
7. Cruzamento final	12
8. Conclusão.....	13
9. Bibliotecas Utilizadas.....	13

1. Introdução

Devido restrições organizacionais e incompatibilidade entre o sistema tercerizado utilizado pela Saude e o sistema utilizado para faturamento da clinica, um tratamento ETL ad hoc foi implementado.

O relatório consiste na apresentação das dificuldades, alguns detalhes técnicos, como o fluxo do código. Para finalizar será listado as obserções e exceções a respeito das decisões tomadas.

2. Contextualização do Problema

A extração, tratamento, manipulação foi realizada com sucesso. Entretanto, um problema de regra de negócio impossibilitou o andamento e conclusão.

Pode ser observado que o relatório de solicitações da saúde possui um período extenso, especificamente de 01/02/2025 a 31/08/2025, o motivo é devido ao relatório de fechamento mensal da empresa não ser respectivo ao mesmo mês no relatório de solicitações da saúde. Explicando melhor, solicitações de um mês não necessariamente são realizadas e faturadas no mesmo mês. Devido a trâmites logísticos, administrativos e tempo de análise laboratorial. Podendo um exame solicitado no mês de agosto ser faturado no mês de outubro. A solução foi aumentar o espaço de tempo do relatório de faturamento da empresa do mês de agosto sendo comparado a solicitações desde fevereiro do mesmo ano para cobrir esse desalinhamento temporal.

Entretanto, um novo problema que inviabilizou a comparação foi a falta de uma chave individual para cada solicitação, tendo em vista que o mesmo paciente pode ter tido 1 ou mais solicitações nesse período de 6 meses, alguns casos possuem solicitações semanais. Sendo quase impossível descobrir qual solicitação é respectiva à remessa de exames faturados naquele fechamento.

A alternativa seria incluir o número da guia de solicitação individual no relatório de faturamento da empresa, utilizando como pivot para o cruzamento de dados. Não sendo possível por questões contratuais.

2.1 Redundancia de Nomenclaturas

A nomenclatura dos exames cadastrados no Sistema da Clinica podem ser diferente do Sistema da Saude. Trazendo um problema ao bater as informações.

Solução

Criado tabela para normalização de acordo com uma chave. Exemplo:

CHAVE	CLINICA	SIS_1	SIS_2
01	ACIDO FOLICO	ACIDO F	ACID FOL.

Exames com nomes diferentes solicitados pelo sistema da saúde podem ser o mesmo, criando uma redundancia, por isso a segunda coluna. (Motivo: Os exames são advindos do SIGTAP, os médicos selecionam um específico, porém a clinica fatura generalizando, ou devido o exame pertencer a um grupo)

2.2 Mais de 1 exame na mesma linha

Contexto/Problema

Solicitação do sistema possibilitou dois exames juntos, provavelmente se complementam, atrapalhando a solução linha x coluna utilizando chave em comum. Exemplo:

CLINICA	SAUDE
TSH	DOSAGEM DE TSH
T4 LIVRE	DOSAGEM DE T4 LIVRE
?	DOSAGEM DE T4 LIVRE e TSH

Solução

Exames compostos são desmembrados e relacionados com seu respectivo código na tabela padrão.

2.3 Grande quantidade de alergen

Contexto/Problemas

As nomenclaturas dos exames tanto o Sistema da Saúde quanto no faturamento da clinica se diferem, não seguindo um padrão e são diversos, talvez centenas.

Solução

A identificação no arquivo da clinica pode ser feita pelo valor fixo “R\$ 10,53”, porém não existe nenhum parametro fixo no lado do Sistema da Saude. Logo um autoincremento pode ser implementado, atualmente é adicionado manualmente na tabela padrão por meio de um tratamento na função `extract_clinica`.

3. Fluxo

Tabelas:

`antes_normaliza_saude.xlsx` -> Após extração do pdf

`depois_normaliza_saude.xlsx` -> Depois de normalizar a tabela

`planilha_clinica.xlsx` -> Após extração do pdf

`dados_cruzado.xlsx` -> Após cruzamento com os nomes

`log_saude.xlsx` -> Após cruzamento com a tabela padrão

`log_faturamento.xlsx` -> Após cruzamento com a tabela padrão

`padrao.xlsx` -> Tabela padronizada dos exames

Scripts:

`busca.py` -> busca o nome do `depois_normaliza_saude` no `planilha_clinica`

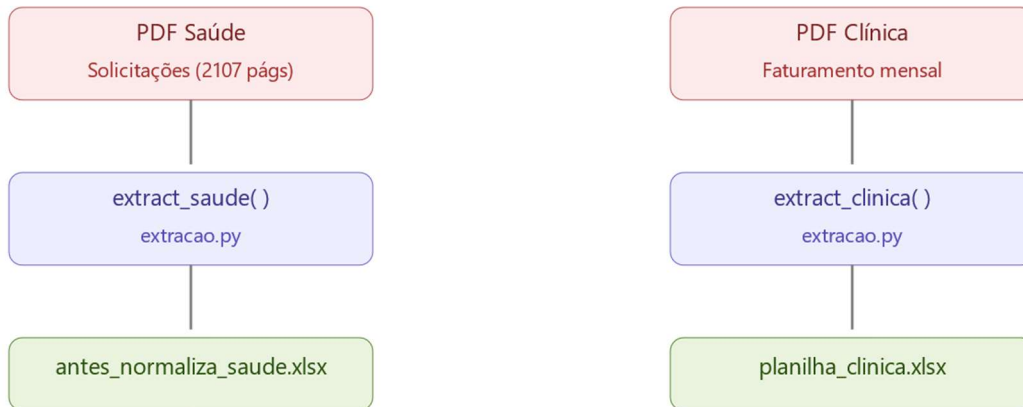
`extracao.py` -> extração dos pdfs

`normalizacao.py` -> normaliza tabela `antes_normaliza_saude`

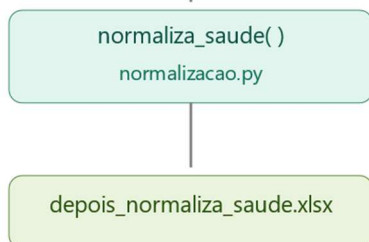
`validacao.py` -> padroniza de acordo com a tabela `padrao` e trata algumas exceções

PDF .xlsx extracao.py normalizacao.py busca.py validacao.py

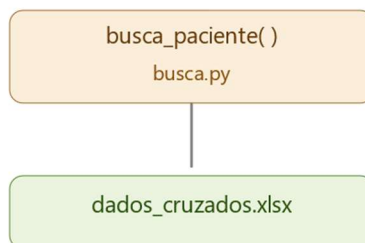
Estágio 1: Extração



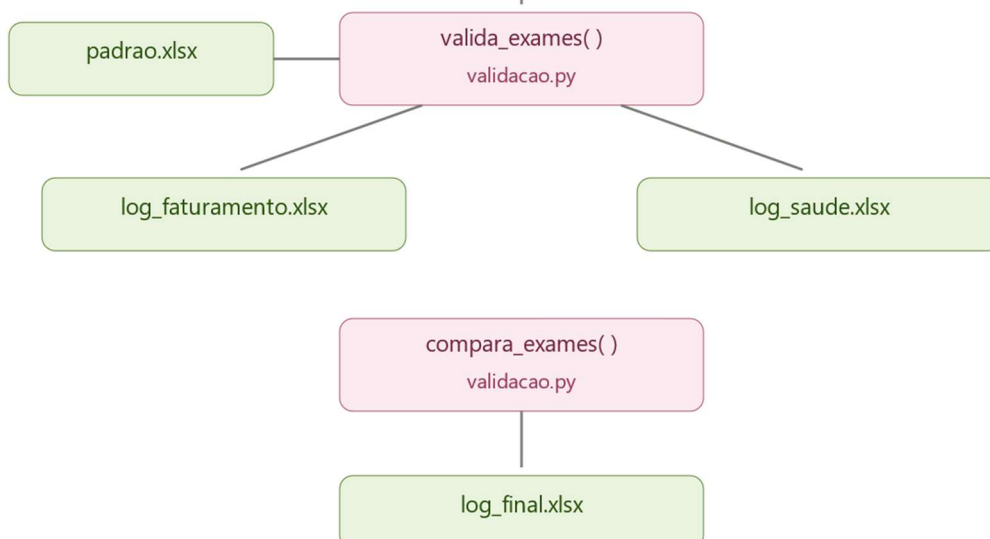
Estágio 2: Normalização



Estágio 3: Cruzamento



Estágio 4: Validação



4. Extração e Tratamento

3.1 Extração Saúde

PDF da Saúde:

Cabeçalho				
Data	Paciente	CPF	Telefone	Exame
dd/mm/yyyy	Fulano1	XXX.XXX.XXX-XX	(XX) XXXXX-XXX	Nome1
dd/mm/yyyy	Fulano1	XXX.XXX.XXX-XX	(XX) XXXXX-XXX	Nome2

Layout: 2107 Páginas x 46 linhas cada = 96.922 Linhas (01/Jun a 15/Set)

Contexto

PDF da Saúde foi mais simples por já ter um formato de tabela. Recortado apenas o cabeçalho e rodapé. O layout do PDF repetia o nome dos pacientes para cada exame solicitado na data respectiva, sendo redundante, como o exemplo acima.

A extração e carregamento para o Excel seguiu o mesmo padrão devido à inviabilidade de tratamento durante esse processo por possuir grande quantidade de linhas.

Descrição Geral:

Percorre todas as páginas de um PDF do tipo "Saúde", extraíndo registros de pacientes contendo data, nome e exame realizado. Retorna uma lista de dicionários, cada um representando um registro de exame.

Fluxo de execução:

- Iteração página a página**
 - Para cada página, obtém o texto completo e o divide em linhas.
 - Linhas de cabeçalho e rodapé são ignoradas usando `linhas[16:-3]`.
- Identificação de paciente**
 - A primeira linha relevante contém **data e nome do paciente**:
 - `data = linha[:10]` → captura os 10 primeiros caracteres (formato `dd/mm/aaaa`).
 - `nome = linha[10:]` → captura o restante da linha como nome.
 - Linhas seguintes com CPF e telefone são ignoradas automaticamente, pois não começam com letras (`.isalpha()`).
- Captura de exame**
 - Linhas que começam com letra (`linha[0].isalpha()`) são consideradas nomes de exames.
 - Para cada exame, cria-se um dicionário:

```
{"Data": data, "Nome": nome, "Exame": exame}
```

Este dicionário é adicionado à lista principal linhas_df. Que é transformado em DataFrame.

3.2 Normalização desse DataFrame:

A função normaliza_saude() faz com que cada solicitação (Data e Nome) fiquem em apenas uma linha, juntando os exames em lista na terceira coluna.

1. **Percorre cada linha do DataFrame de entrada (doc).**
2. **Agrupa as informações por paciente e data:**
 - Usa uma chave (nome, data) para diferenciar, porque o mesmo paciente pode ter solicitações em dias diferentes.
3. **Constrói um dicionário com listas de exames para cada paciente/data.**
 - Cada entrada tem:

```
{"Data": data, "Nome": nome, "Solicitação": numero "Exames":  
lista_de_exames }
```

Importante: A coluna solicitação serve para visualização mais fácil de qual solicitação poderia estar sendo faturado. Não foi usada para fins práticos.

Tabela inicial:

antes_normaliza_saude.xlsx

Data	Nome	Exame
01/01/2022	Joao Silva	Exame 1
01/01/2022	Joao Silva	Exame 2

Tabela final normalizada:

depois_normaliza_saude.xlsx

Data	Nome	Solicitação	Exames
01/01/2022	Joao Silva	1	["Exame 1", "Exame 2", "Exame 2"]
02/01/2022	Maria Ribeiro	1	["Exame 1", "Exame 2", "Exame 3"]

3.3 Extração Clínica

PDF da Clínica:

Cabeçalho			
dd/mm/yyyy	Código	Paciente	Valor R\$
		Exame1	9,99
		Exame2	9,99

dd/mm/yyyy	Código	Paciente	Valor R\$

Layout: Numero de páginas e linhas imprevisíveis (Individual por UBS e imprevisível média de solicitações e exames mensais).

Contexto

O pdf da Clínica foi mais complexo devido a necessidade de delimitação na extração, por exemplo, identificar onde começa e onde termina o faturamento individual de cada paciente, ocorrendo quebras de páginas diferentes a cada página. Porém a saída já sai normalizada em uma única linha por paciente.

Descrição geral:

Percorre todas as páginas de um PDF clínico, linha a linha, identificando blocos de informações de pacientes e extraíndo suas datas, nomes e exames realizados. Retorna uma lista de dicionários, cada um representando um paciente.

Fluxo de execução:

1. **Iteração página a página**
 - Para cada página, obtém o texto completo e o divide em linhas.
2. **Identificação do paciente**
 - **Data:** reconhecida por regex no formato dd/mm/aaaa.
 - **Nome:** localizado duas linhas abaixo da data.
3. **Bloco de exames**
 - **Início:** detectado quando a linha começa com "VALOR R\$".
 - **Término:** identificado pela linha de traços "---".
 - Durante o bloco, as linhas são processadas para diferenciar exames de cabeçalhos/valores.
4. **Cabeçalhos e valores numéricos**
 - Uma tupla de cabeçalhos (ex.: "QTD", "TOTAL", "LAB") serve como filtro para ignorar rodapés ou trechos não relacionados.
 - Linhas que começam com números (valores) também são descartadas como possíveis nomes de exame.

5. Nomes de exames

- Linhas que não se enquadram nos critérios acima são consideradas descrições de exames e adicionadas à lista.
- Caso seja detectado um valor específico (ex.: "10,53"), o texto imediatamente anterior é armazenado em uma lista de alergias.

6. Acúmulo final

- Ao encerrar o bloco de exames ("--"), o dicionário do paciente é construído com:

```
{"Data": data, "Nome": nome, "Exame": exames}
```

Esse dicionário é anexado à lista principal paciente.

Saída Final:

planilha_clinica.xlsx

Data	Nome	Exame
01/01/2022	Joao Silva	["Exame 1", "Exame 2", "Exame 2"]
02/01/2022	Maria Ribeiro	["Exame 1", "Exame 2", "Exame 3"]

5. Verificação paciente faturado x solicitações

Contexto e Descrição Geral:

Após extraído e normaliza as planilhas é comparada as duas bases de dados de pacientes (base_clinica e base_saude) por meio da **busca_saude()** e gera um log indicando se cada paciente da clínica está presente na base de saúde. Além disso, registra os exames de cada base e cria uma planilha Excel com o resultado.

IMPORTANTE: A chave de comparação é apenas o nome do paciente. Condição esta que, é a complicação principal citada no inicio desse documento, a qual ocasionou o encerramento devido resultado pouco aproveitável e não adequado.

Fluxo de execução:

1. Normalização de nomes

- Cria uma nova coluna nome_limpo em ambas as bases, aplicando a função limpar() para padronizar os nomes (remoção de acentos, espaços extras, etc.).

2. Preparação da base de saúde

- Ordena alfabeticamente os nomes da base de saúde.
- Converte os nomes em uma lista (nomes_saude) para permitir busca binária (bisect), garantindo maior eficiência.

3. Iteração sobre pacientes da base clínica

Para cada paciente (linha) da base clínica:

- Obtém nome_limpo e lista de exames (Exame).
- Utiliza bisect.bisect_left para localizar o índice do nome na lista nomes_saude.
- Se o nome estiver presente:
 - Recupera os exames correspondentes da base de saúde (Exames_saude).
 - Adiciona um registro no log com Status = "Encontrado".
- Caso contrário:
 - Adiciona um registro no log com Status = "Não encontrado" e Exames_saude = None.

4. Criação do df

- Constrói um DataFrame dados_cruzados contendo:

dados_cruzados.xlsx

Data	Paciente	Exames_clinica	Exames_saude	Status
01/01/2022	Joao Silva	["Exame 1", "Exame 2", "Exame 2"]	["Exame 1", "Exame 2", "Exame 3"]	Encontrado
02/01/2022	Maria Ribeiro	["Exame 1", "Exame 2", "Exame 3"]	["Exame 1", "Exame 2", "Exame 3"]	Não Encontrado

IMPORTANTE: Destaque que algumas linhas ganharam status como "Não Encontrado" por que seus nomes estavam escritos de forma diferente nos dois relatórios. Erros simples. Sucetível a correção manual.

6. Comparação dos exames

Contexto:

Após o cruzamento de dados, a função **valida_exames()** é chamada com a dados_cruzados como parametro. Ao finalizado, filtra os pacientes faturados, seus exames faturados, e exames solicitados.

Descrição Geral:

As duas tabelas terão seus exames padronizados de acordo com a tabela padrao.xlsx, transformando seus exames em códigos-chave. Se não existir o exame na tabela padrão, ele não será removido, mas continuara em texto

Fluxo de execução:

1. Carregamento da tabela de padronização

- Lê data/padrao.xlsx, que contém a relação entre nomenclaturas da clínica (coluna CLINICA), do sistema da saúde (colunas SIS_1 e SIS_2) e uma chave numérica identificadora (CHAVE).

2. Iteração sobre os dados cruzados

- Para cada linha do DataFrame dados_cruzados, converte as colunas Exames_Clinica e Exames_Saude de string para lista utilizando str_para_list().
3. **Resolução dos exames da clínica**
- Para cada exame da lista da clínica, percorre a tabela de padronização buscando correspondência na coluna CLINICA (que pode conter mais de um nome por linha, ex.: ['PPF1º...', 'PPF2º...', 'PPF3º...']).
 - Se encontrado, registra no log_faturamento com a chave numérica correspondente.
 - Se não encontrado, registra o próprio nome do exame no lugar da chave, sinalizando item sem correspondência na tabela padrão.
4. **Resolução dos exames da saúde**
- Segue a mesma lógica, porém busca correspondência nas colunas SIS_1 e SIS_2.
 - **Tratamento de exames compostos:** Antes da busca genérica, verifica dois casos especiais onde o sistema da saúde agrupa dois exames em uma única solicitação:
 - DOSAGEM DE TSH E T4 LIVRE → desmembrado em duas entradas separadas: chave do TSH e chave do T4 LIVRE.
 - DOSAGEM DE PROTEINAS TOTAIS E FRACOES → desmembrado em: chave de PROTEINAS TOTAIS e chave de PROTEINAS FRACOES.
 - Exames compostos são tratados via continue, pulando a busca genérica.
 - Exames não encontrados na tabela padrão são registrados com o nome original, assim como no lado da clínica.
5. **Exportação dos logs**
- log_faturamento → output/log_faturamento.xlsx
 - log_saude → output/log_saude.xlsx

log_faturamento:

Data	Nome	Solicitação	Exames
01/01/2022	Joao Silva	1	[1 , 10, 20, 35]
02/01/2022	Maria Ribeiro	1	[75, 93]

log_saude:

Data	Nome	Exames
01/01/2022	Joao Silva	[1 , 10, 20, 35]
02/04/2022	Joao Silva	[1, 13]
02/04/2022	Maria Ribeiro	[5, 93]

Note a redundancia no log_saude, devido o espaço de tempo do relatório de saúde.

7. Cruzamento final

Carregamento e cruzamento dos dois logs normalizados com o código padrão, apresentando exames que não foram solicitados pela saúde mas que foram faturados.

log_final:

Nome	Status	Exames_Nao_Solicitados
Adelio	OK	[]
Cleber	Faturamento Errado	[1, 13, 51, 98]
Joao	Faturamento Errado	[5, 22, 54, 97]

A coluna de exames não solicitados aponta quais estão sobrando usando teoria de conjuntos.

Descrição geral:

Cruza usando novamente o nome como chave e compara quais exames foram faturados mas não pedidos, usando teoria de conjuntos (não foi usado sets). É criado então uma coluna com o Status OK ou Faturamento Errado.

Fluxo de execução:

- Carregamento dos logs**
 - Lê output/log_saude.xlsx e output/log_faturamento.xlsx, gerados pela função valida_exames.
- Iteração a partir da base da saúde**
 - Para cada paciente no log da saúde, percorre o log da clínica buscando correspondência pelo campo Nome.
 - Ao encontrar o primeiro match, converte as colunas de exames de ambos os lados para lista via str_para_list().
 - O break encerra a busca no primeiro match encontrado.
- Verificação de divergência**
 - Calcula a diferença: exames presentes no faturamento da clínica que **não** constam nas solicitações da saúde.
 - Se existem exames sem correspondência → status "Faturamento Errado", com a lista dos exames divergentes em Exames_Nao_Solicitados.
 - Se todos os exames faturados possuem correspondência → status "OK".
- Exportação**
 - Gera output/log_final.xlsx com o resultado.

Entretanto devido a grande quantidade de solicitações do mesmo paciente sem uma chave única, não há possibilidade de uma conclusão.

Uma possibilidade que iniciei mas não dei continuidade é de comparar as listas dos códigos da coluna exames_clinica com a coluna exames_saude e selecionar a mais semelhante. Porém podem haver duas solicitações de exames com os mesmo exames.

8. Conclusão

A conclusão é inviabilidade devido desalinhamento temporal e falta de chave individual para cruzar as tabelas.

9. Bibliotecas Utilizadas

- **pymupdf (fitz)**
Usada para leitura dos arquivos PDF.
Permite extrair o texto página por página, linha por linha, que foi a base para capturar datas, nomes e exames.
(Ex: `pagina.get_text("text")`)
- **unicodedata**
Utilizada para normalização de texto.
Remove acentos e caracteres especiais, garantindo que nomes de pacientes sejam comparados de forma consistente.
(Ex: `unicodedata.normalize("NFKD", texto)`).
- **re (expressões regulares)**
Usada para identificar padrões dentro do texto extraído, como a detecção da data no formato dd/mm/aaaa.
(Ex: `re.compile(r"\d{2}/\d{2}/\d{4}")`).
- **time**
Auxiliar em marcação de tempo de execução e eventuais pausas.
Pode ser útil para medir performance ou evitar travamentos em loops pesados.
(Foi importada duas vezes, pode ser consolidado em um só import.)
- **bisect**
Utilizada para busca binária em listas ordenadas.
Permitindo verificar rapidamente se um paciente da base_clínica existe na base_saúde.
(Ex: `bisect_left(nomes_saude, nome)`).
- **ast**
Usada para avaliar/transformar strings em estruturas Python quando necessário (ex: converter string de lista em objeto real).
Ainda pouco usada no fluxo principal, mas útil se a leitura de planilhas retornar listas como string.
- **pandas**
Biblioteca central para manipulação de dados.
Utilizada para armazenar, transformar e exportar as informações extraídas dos PDFs.
(Ex: criação de DataFrame com `pd.DataFrame()` e exportação to_excel).
- **functools.wraps**
Importada para auxiliar na criação de decorators preservando metadados da função original (nome, docstring).

Ainda não foi usada diretamente, mas pode ser útil se for criar funções de logging ou medir tempo de execução via decorator.